Sprint 26 Structured Notes: Asynchronous JavaScript, Promises, Fetch, Event Loop, and Geolocation

0. What Sprint 26 Is About

Sprint 26 teaches you how JavaScript handles work that does **not finish immediately**.

Before Sprint 26, most of your code was easy to read from top to bottom:

```
const total = 10 + 5;
console.log(total);
```

JavaScript runs the first line, then the second line.

Sprint 26 changes the mental model.

Some JavaScript work starts now but finishes later.

Examples:

```
fetch data from a server
wait for a timer
ask for the user's location
load a file
wait for a slow network response
```

Main goal:

> Start slow work, keep the page responsive, wait for results safely, handle failures, and update the page when async work finishes.

By the end of Sprint 26, you should understand:

- synchronous code
- asynchronous code
- why async code exists
- callbacks in async code
- `setTimeout()` review
- Promises

- Promise states
- `.then()`
- `.catch()`
- `.finally()`
- `async`
- `await`
- Fetch API
- `response.json()`
- `response.ok`
- safer Fetch patterns
- loading, success, and error states
- HTTP methods
- CRUD and HTTP method mapping
- sending JSON with `JSON.stringify()`
- `async` and `defer` script loading
- JavaScript engine vs runtime
- event loop basics
- microtasks
- macrotasks/tasks
- Promise tools: `Promise.all()`, `Promise.allSettled()`, and `Promise.race()`
- Geolocation API
- permission-denial handling
- common beginner mistakes
- practice tasks and checkpoints

Part A: Current Progress Before Sprint 26

1. What You Already Know

Sprint 26 comes after you already know a large part of JavaScript.

You have learned the core JavaScript layer:

Area	What You Learned
Values and variables	Store data with `let` and `const`
Data types	Work with strings, numbers, booleans, objects, arrays, `null`, and `undefined`
Strings	Create, search, slice, trim, replace, and format text
Numbers and booleans	Calculate, compare, and make decisions
Functions	Write reusable logic with parameters and return values

Area	What You Learned
Arrays	Store ordered lists
Loops	Repeat work safely
Objects	Store structured key-value data
Callbacks	Pass functions into other functions
Sets and Maps	Use special collection tools
Classes	Create reusable object blueprints
Regex	Search and validate text with patterns
Debugging	Find and fix problems systematically

You have also learned the browser JavaScript layer:

Area	What You Learned
DOM selection	Find page elements with JavaScript
DOM updates	Change text, attributes, classes, styles, and children
Events	Respond to clicks, typing, form submission, and timers
Accessibility	Keep visual state and accessibility state in sync
Forms	Validate user input and control submission
Dates and media	Use browser APIs for time, audio, video, and media
CRUD and storage	Save simple data with `localStorage` and `sessionStorage`

You also moved into algorithm thinking:

Area	What You Learned
Big O	Reason about time and space growth
Searching and sorting	Find and organize data
Recursion	Write functions that call themselves
Data structures	Use stacks, queues, linked lists, graphs, trees, heaps, and DP
Dynamic programming	Save repeated answers to avoid repeated work

Simple summary:

> Before Sprint 26, you learned how to write JavaScript, control the DOM, and solve algorithm problems. Sprint 26 teaches you how JavaScript handles work that finishes later.

2. The Main Shift in Sprint 26

Before Sprint 26, you often read code like this:

```
console.log("A");
console.log("B");
console.log("C");
```

Output:

```
A
B
C
```

That code is synchronous.

It runs in order.

But Sprint 26 introduces code like this:

```
fetch("/users.json")
  .then(response => response.json())
  .then(data => console.log(data));

console.log("This may run before the data arrives");
```

The `fetch()` request starts, but the data may arrive later.

So this line may run before the data is ready:

```
console.log("This may run before the data arrives");
```

Main Sprint 26 idea:

> Some work starts now, finishes later, and needs special code to handle the result.

Part B: Synchronous and Asynchronous Code

3. Synchronous Code

Synchronous code runs one line at a time.

Each line must finish before the next line starts.

Example:

```
console.log("A");
console.log("B");
console.log("C");
```

Output:

```
A
B
C
```

Step-by-step:

Step	Code	What Happens
1	`console.log("A")`	Prints `A`
2	`console.log("B")`	Prints `B`
3	`console.log("C")`	Prints `C`

Simple meaning:

> Synchronous code runs in a predictable top-to-bottom order.

---

4. Asynchronous Code

Asynchronous code allows slow work to happen without freezing the whole page.

Example:

```
console.log("Start");
```

```
setTimeout(() => {
  console.log("Timer finished");
}, 1000);

console.log("End");
```

Output:

```
Start
End
Timer finished
```

This may look strange at first.

Why does `End` print before `Timer finished`?

Because `setTimeout()` starts a timer, but JavaScript does not stop the whole program while waiting.

Simple meaning:

> Asynchronous code lets JavaScript start a slow task, continue running other code, and come back when the slow task is finished.

---

5. Why Asynchronous Code Exists

Some tasks are slow.

Examples:

- asking a server for data
- waiting for a timer
- loading an image
- requesting the user's location
- reading data from a file
- waiting for a network response

If JavaScript blocked the whole page during these tasks, the user experience would be bad.

Bad mental model:

```
Ask server for data
Freeze page
Wait
```

```
Wait
Wait
Data arrives
Page works again
```

Better async model:

```
Ask server for data
Keep page usable
Data arrives later
Run code that handles the data
```

Beginner rule:

> Async JavaScript keeps the browser responsive while slow work happens.

Part C: Callback Review

6. What Is a Callback?

A callback is a function passed into another function.

Example:

```
function runTask(callback) {
  callback();
}

function sayHello() {
  console.log("Hello");
}

runTask(sayHello);
```

Output:

```
Hello
```

Here:

```
sayHello
```

is passed into:

```
runTask
```

Then `runTask()` calls it here:

```
callback();
```

Simple meaning:

> A callback is a function you give to another function so it can run later.

---

7. Callbacks in Async Code

Async APIs often use callbacks because they need to run your function later.

Example with `setTimeout()`:

```
setTimeout(() => {
  console.log("This runs later");
}, 1000);
```

The callback is:

```
() => {
  console.log("This runs later");
}
```

JavaScript does not run it immediately.

It runs it after the timer finishes.

Simple meaning:

> Async callbacks are functions that wait until something happens.

---

8. Passing a Function vs Calling a Function

This is a common beginner issue.

Correct:

```
setTimeout(sayHello, 1000);
```

This means:

```
Give the sayHello function to setTimeout so it can run later.
```

Usually wrong:

```
setTimeout(sayHello(), 1000);
```

This means:

```
Run sayHello immediately, then pass its return value to setTimeout.
```

If `sayHello()` returns `undefined`, then you are basically doing this:

```
setTimeout(undefined, 1000);
```

Beginner rule:

> When an async API expects a callback, usually pass the function without parentheses.

Part D: Promises

9. What Is a Promise?

A `Promise` represents asynchronous work that may finish later.

Simple meaning:

> A Promise is an object that says: "I do not have the answer yet, but I will later."

Example idea:

```
You order food.
The restaurant gives you a receipt.
The receipt is not the food.
But it represents food that should arrive later.
```

A Promise is similar.

It is not the final result yet.

It represents a future result.

---

10. Promise States

A Promise can be in one of three states.

State	Meaning
`pending`	Still waiting
`fulfilled`	Finished successfully
`rejected`	Failed

Simple mental model:

```
pending   -> still working
fulfilled -> success
rejected  -> failure
```

---

11. Creating a Basic Promise

Example:

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data loaded");
  }, 1000);
});
```

```
promise.then(result => {
  console.log(result);
});
```

Output after about 1 second:

```
Data loaded
```

What happens:

Code	Meaning
`new Promise(...)`	Create a Promise
`resolve("Data loaded")`	Mark the Promise as successful
`.then(...)`	Run code when the Promise succeeds
`result`	The value passed to `resolve()`

---

12. `resolve` and `reject`

Inside a Promise:

```
resolve(value)
```

means the async work succeeded.

```
reject(error)
```

means the async work failed.

Example:

```
const promise = new Promise((resolve, reject) => {
  const success = true;

  if (success) {
    resolve("It worked");
  } else {
    reject(new Error("It failed"));
```

```
  }
});
```

Simple meaning:

> Use `resolve()` for success and `reject()` for failure.

Part E: `.then()`, `.catch()`, and `.finally()`

13. `.then()`

Use `.then()` when a Promise succeeds.

Example:

```
const promise = Promise.resolve("Hello");

promise.then(value => {
  console.log(value);
});
```

Output:

```
Hello
```

Simple meaning:

> `.then()` runs after a Promise is fulfilled.

---

14. `.catch()`

Use `.catch()` when a Promise fails.

Example:

```
const promise = Promise.reject(new Error("Something went wrong"));

promise.catch(error => {
```

```
  console.error(error.message);
});
```

Output:

```
Something went wrong
```

Simple meaning:

`.catch()` handles Promise errors.

---

15. `.finally()`

Use `.finally()` for code that should run whether the Promise succeeds or fails.

Example:

```
fetch("/users.json")
  .then(response => response.json())
  .then(users => {
    console.log(users);
  })
  .catch(error => {
    console.error("Something failed:", error.message);
  })
  .finally(() => {
    console.log("Request finished");
  });
```

Simple meaning:

```
Try to fetch users.
If it works, parse JSON.
If that works, use the users.
If anything fails, catch the error.
Always run finally at the end.
```

Common uses for `.finally()`:

- hide a loading spinner
- enable a button again

- clear temporary status
- log that the request finished

Beginner rule:

> Use `.finally()` for cleanup that should happen either way.

---

16. Promise Chain

A Promise chain connects multiple async steps.

Example:

```
fetch("/users.json")
  .then(response => response.json())
  .then(users => {
    console.log(users);
  })
  .catch(error => {
    console.error(error.message);
  });
```

Step-by-step:

Step	Code	Meaning
1	`fetch("/users.json")`	Ask for data
2	`.then(response => response.json())`	Convert response body into JavaScript data
3	`.then(users => ...)`	Use the parsed users
4	`.catch(error => ...)`	Handle any error from the chain

Simple meaning:

> Promise chains let you describe async steps in order.

Part F: `async` and `await`

17. What Is `async`?

`async` marks a function as asynchronous.

Example:

```
async function loadUsers() {
  return "Users loaded";
}
```

An `async` function always returns a Promise.

Example:

```
async function sayHello() {
  return "Hello";
}

const result = sayHello();
console.log(result);
```

Conceptual output:

```
Promise { "Hello" }
```

To get the value, use `await` or `.then()`.

```
sayHello().then(value => {
  console.log(value);
});
```

Output:

```
Hello
```

Beginner rule:

> An `async` function returns a Promise, even if it looks like it returns a normal value.

---

18. What Is `await`?

`await` waits for a Promise to settle inside an `async` function.

Example:

```
async function loadUsers() {
  const response = await fetch("/users.json");
  const users = await response.json();

  console.log(users);
}

loadUsers();
```

Simple meaning:

```
Wait for fetch to finish.
Then wait for JSON parsing to finish.
Then use the users.
```

Important rule:

- `await` pauses inside the `async` function, not the whole webpage.

---

19. `.then()` vs `async/await`

Promise chain version:

```
fetch("/users.json")
  .then(response => response.json())
  .then(users => {
    console.log(users);
  });
```

`async/await` version:

```
async function loadUsers() {
  const response = await fetch("/users.json");
  const users = await response.json();

  console.log(users);
}
```

```
loadUsers();
```

Both do the same basic job.

Simple comparison:

Style	Meaning
`.then()`	Handle Promise result with chained callbacks
`async/await`	Write Promise code in a style that looks more top-to-bottom

Beginner rule:

> Use `async/await` when it makes async code easier to read.

---

20. Important `await` Execution Order

Study this code:

```
async function loadUsers() {
  console.log("Inside start");

  const response = await fetch("/users.json");
  const users = await response.json();

  console.log(users);
}

console.log("Before");
loadUsers();
console.log("After");
```

Likely output:

```
Before
Inside start
After
users data later
```

Why?

1. `Before` runs first.
2. `loadUsers()` starts.
3. `Inside start` runs.
4. `await fetch(...)` pauses inside `loadUsers()`.
5. JavaScript continues outside the function.
6. `After` runs.
7. When data arrives, the rest of `loadUsers()` continues.

Simple meaning:

> `await` does not freeze all JavaScript. It only pauses the current async function.

Part G: Fetch API

21. What Is `fetch()`?

`fetch()` is used to make network requests.

Simple meaning:

> `fetch()` asks for data from a URL or sends data to a URL.

Basic example:

```
fetch("/users.json");
```

This starts a network request.

But by itself, this does not yet give you the final data.

`fetch()` returns a Promise.

---

22. Basic GET Request with `async/await`

Example:

```
async function loadData() {
  const response = await fetch("/data.json");
```

```
  const data = await response.json();

  console.log(data);
}

loadData();
```

Line-by-line:

Code	Meaning
`async function loadData()`	Create an async function
`await fetch("/data.json")`	Wait for the server response
`response.json()`	Parse JSON from the response body
`await response.json()`	Wait for JSON parsing to finish
`console.log(data)`	Inspect the parsed JavaScript data

Simple meaning:

```
fetch("/data.json") -> ask for data
await response -> wait for response
response.json() -> convert JSON into JavaScript data
console.log(data) -> inspect the result
```

---

23. What Is a Response Object?

`fetch()` gives you a response object.

The response object contains information about the server response.

Useful properties and methods:

Property / Method	Meaning
`response.ok`	`true` if the status is successful
`response.status`	HTTP status code, like `200`, `404`, or `500`
`response.json()`	Parse response body as JSON
`response.text()`	Parse response body as plain text

Beginner rule:

> `fetch()` gives you a response first. You usually need `response.json()` to get the actual data.

---

24. What Does `response.json()` Do?

`response.json()` reads JSON from the response body and converts it into JavaScript data.

Example JSON from server:

```
[
  { "name": "Ada" },
  { "name": "Linus" }
]
```

JavaScript after parsing:

```
const users = [
  { name: "Ada" },
  { name: "Linus" }
];
```

Important:

```
response.json()
```

also returns a Promise.

That is why you usually write:

```
const users = await response.json();
```

Simple meaning:

> `response.json()` converts JSON text from the server into usable JavaScript data.

Part H: Safer Fetch Pattern

25. Beginner Fetch Mistake

Beginner code often assumes everything worked:

```
const response = await fetch("/users");
const users = await response.json();
```

Problem:

- What if the URL is wrong?
- What if the server returns `404`?
- What if the server returns `500`?
- What if the network fails?
- What if the response is not valid JSON?

This code has no plan for failure.

---

26. Better Fetch with `try/catch`

Better version:

```
async function loadUsers() {
  try {
    const response = await fetch("/users");

    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }

    const users = await response.json();
    console.log(users);
  } catch (error) {
    console.error("Could not load users:", error.message);
  }
}

loadUsers();
```

This version handles failures.

---

27. Why Use `try/catch`?

`try/catch` lets you handle errors safely.

```
try {
  // Code that might fail
} catch (error) {
  // Code that handles the failure
}
```

Simple meaning:

> Try to do the risky work. If it fails, catch the error and respond safely.

Common async errors:

- network failure
- bad URL
- server problem
- invalid JSON
- rejected Promise

---

28. Why Check `response.ok`?

A server may respond with an error status.

Examples:

Status	Meaning
`200`	OK / success
`201`	Created
`400`	Bad request
`401`	Unauthorized
`403`	Forbidden
`404`	Not found
`500`	Server error

`response.ok` is usually `true` for successful status codes and `false` for error status codes.

Example:

```
if (!response.ok) {
  throw new Error(`Request failed: ${response.status}`);
}
```

Simple meaning:

> Check whether the server response was actually successful before using the data.

---

29. Safer Fetch Template

Use this as a beginner template:

```
async function loadData(url) {
  try {
    const response = await fetch(url);

    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }

    const data = await response.json();
    return data;
  } catch (error) {
    console.error("Load failed:", error.message);
    return null;
  }
}
```

Usage:

```
async function main() {
  const users = await loadData("/users.json");

  if (users === null) {
    console.log("No users loaded.");
    return;
  }

  console.log(users);
}
```

```
main();
```

Beginner rule:

> Async code should have a success path and a failure path.

Part I: Loading, Success, and Error States in the DOM

30. Why UI State Matters

When async work happens, the user should know what is happening.

Common states:

State	Meaning	Example Message
idle	Nothing is happening yet	`Idle`
loading	Request is running	`Loading...`
success	Request worked	`Users loaded.`
error	Request failed	`Could not load users.`

Simple meaning:

> Do not leave the user guessing while async work is happening.

---

31. DOM Example with Loading and Error State

HTML:

```
<button id="loadBtn">Load users</button>
<p id="status" aria-live="polite">Idle</p>
<ul id="users"></ul>
```

JavaScript:

```
const loadBtn = document.getElementById("loadBtn");
const status = document.getElementById("status");
const usersList = document.getElementById("users");

async function loadUsers() {
  try {
    status.textContent = "Loading...";
    usersList.textContent = "";

    const response = await fetch("/users.json");

    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }

    const users = await response.json();

    users.forEach(user => {
      const li = document.createElement("li");
      li.textContent = user.name;
      usersList.appendChild(li);
    });

    status.textContent = "Users loaded.";
  } catch (error) {
    status.textContent = "Could not load users.";
    console.error(error.message);
  }
}

loadBtn.addEventListener("click", loadUsers);
```

This combines earlier sprint skills:

Skill	Used Here
DOM selection	`getElementById()`
Events	`addEventListener()`
Async functions	`async function loadUsers()`
Await	`await fetch(...)`
Fetch	Request user data
Error handling	`try/catch`

Skill	Used Here
Safe DOM updates	`textContent`
Accessibility	`aria-live="polite"`

---

32. Why Use `aria-live` for Status?

Example:

```
<p id="status" aria-live="polite">Idle</p>
```

When JavaScript changes the status text:

```
status.textContent = "Users loaded.";
```

assistive technology can announce the update.

Simple meaning:

> `aria-live` helps dynamic status messages be understood by more users.

Beginner rule:

> Important async status messages should be visible and accessible.

Part J: HTTP Methods and CRUD

33. CRUD Review

CRUD means:

Letter	Meaning	Simple App Example
C	Create	Add a task
R	Read	Show tasks
U	Update	Edit a task
D	Delete	Remove a task

Sprint 17 taught CRUD with arrays and browser storage.

Sprint 26 connects CRUD to server requests.

---

34. HTTP Method Mapping

CRUD action	HTTP method	Meaning
Create	`POST`	Add new data
Read	`GET`	Fetch existing data
Replace/update	`PUT`	Replace existing data
Partial update	`PATCH`	Change part of existing data
Delete	`DELETE`	Remove data

Simple meaning:

> HTTP methods tell the server what kind of action you want.

---

35. GET Request

Use `GET` to read data.

Most basic `fetch()` requests are GET requests by default.

```
async function getTasks() {
  const response = await fetch("/tasks");

  if (!response.ok) {
    throw new Error(`Read failed: ${response.status}`);
  }

  return response.json();
}
```

Simple meaning:

> GET asks the server for existing data.

---

36. POST Request

Use `POST` to create new data.

Example:

```
async function createTask(task) {
  const response = await fetch("/tasks", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify(task)
  });

  if (!response.ok) {
    throw new Error(`Create failed: ${response.status}`);
  }

  return response.json();
}
```

Usage:

```
createTask({
  text: "Study async JavaScript",
  done: false
});
```

Simple meaning:

> POST sends new data to the server.

---

37. Why Use `JSON.stringify()`?

JavaScript object:

```
const task = {
  text: "Study",
  done: false
};
```

Network request bodies usually send text.

So you convert the object into JSON text:

```
JSON.stringify(task)
```

Result:

```
{"text":"Study","done":false}
```

Simple meaning:

> `JSON.stringify()` converts JavaScript data into JSON text so it can be sent in a request body.

---

38. Headers with JSON

When sending JSON, use this header:

```
headers: {
  "Content-Type": "application/json"
}
```

Simple meaning:

> This tells the server: “I am sending JSON.”

---

39. PUT Request

Use `PUT` to replace existing data.

Example:

```
async function replaceTask(id, task) {
  const response = await fetch(`/tasks/${id}`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
```

```
    body: JSON.stringify(task)
  });

  if (!response.ok) {
    throw new Error(`Replace failed: ${response.status}`);
  }

  return response.json();
}
```

Simple meaning:

> PUT usually replaces the whole resource.

---

40. PATCH Request

Use `PATCH` to update part of existing data.

Example:

```
async function updateTaskStatus(id, done) {
  const response = await fetch(`/tasks/${id}`, {
    method: "PATCH",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ done: done })
  });

  if (!response.ok) {
    throw new Error(`Update failed: ${response.status}`);
  }

  return response.json();
}
```

Simple meaning:

> PATCH changes only part of a resource.

---

41. DELETE Request

Use `DELETE` to remove data.

Example:

```
async function deleteTask(id) {
  const response = await fetch(`/tasks/${id}`, {
    method: "DELETE"
  });

  if (!response.ok) {
    throw new Error(`Delete failed: ${response.status}`);
  }

  return true;
}
```

Simple meaning:

> DELETE asks the server to remove something.

Part K: Script Loading with `async` and `defer`

42. Why Script Loading Matters

HTML is parsed from top to bottom.

JavaScript files can affect how the browser loads the page.

Example:

```
<script src="app.js"></script>
```

A normal script can block HTML parsing while it loads and runs.

This can cause problems if the script tries to select HTML that has not been parsed yet.

---

43. Normal Script

Example:

```
<script src="app.js"></script>
```

Simple meaning:

> Load and run the script when the browser reaches this line.

Possible issue:

```
<script src="app.js"></script>
<button id="btn">Click</button>
```

If `app.js` tries to select `#btn`, the button may not exist yet.

---

44. `defer`

Example:

```
<script src="app.js" defer></script>
```

`defer` means:

- download the script while HTML parsing continues
- run the script after HTML parsing finishes
- preserve script order

Good default for beginner browser apps:

```
<script src="app.js" defer></script>
```

Simple meaning:

> Use `defer` when your script depends on the HTML being ready.

---

45. `async`

Example:

```
<script src="analytics.js" async></script>
```

`async` means:

- download the script while HTML parsing continues
- run the script as soon as it finishes downloading
- does not guarantee order with other async scripts

Good for:

- analytics scripts
- ads scripts
- independent scripts
- scripts that do not depend on DOM order

Simple meaning:

> Use `async` for independent scripts where execution order does not matter.

---

46. `async` vs `defer`

Attribute	When It Runs	Order Guaranteed?	Good For
normal script	Immediately when reached	Yes, but can block parsing	Small/simple scripts
`defer`	After HTML parsing	Yes	Main app scripts
`async`	As soon as downloaded	No	Independent scripts

Beginner rule:

> For your main app JavaScript file, use `defer` most of the time.

Part L: JavaScript Engine and Runtime

47. JavaScript Engine

The JavaScript engine reads and executes JavaScript code.

Simple meaning:

> The engine is the part that understands and runs JavaScript.

Examples of JavaScript engines:

- V8 in Chrome and Node.js
- SpiderMonkey in Firefox
- JavaScriptCore in Safari

You do not need to memorize engine names as a beginner, but you should know the concept.

---

48. JavaScript Runtime

The runtime is the environment around the engine.

In the browser, the runtime gives JavaScript extra tools like:

```
document
window
navigator
setTimeout
fetch
localStorage
```

These are not all core language syntax.

Many are browser-provided APIs.

Simple meaning:

> The runtime gives JavaScript extra powers depending on where the code runs.

---

49. Engine vs Runtime

Term	Simple Meaning	Example Responsibility
JavaScript engine	Runs JavaScript code	Executes functions and expressions
Runtime	Provides environment tools	Gives APIs like `fetch()` and `setTimeout()`

Example:

```
fetch("/data.json");
```

`fetch()` is provided by the browser runtime.

Example:

```
const total = 2 + 3;
```

The JavaScript engine executes this calculation.

Beginner rule:

> The engine runs JavaScript. The runtime provides extra APIs.

Part M: Event Loop Basics

50. Why the Event Loop Matters

JavaScript usually runs one main piece of code at a time.

But async work can finish later.

The event loop helps JavaScript decide when to run waiting callbacks.

Simple meaning:

> The event loop coordinates normal code, Promise callbacks, timers, and other async work.

---

51. Synchronous Code Runs First

Example:

```
console.log("A");
console.log("B");
```

Output:

```
A
B
```

Synchronous code runs first before waiting async callbacks.

---

52. Timer Callback Example

Example:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output:

```
A
C
B
```

Why does `B` come last even though the timer is `0`?

Because `setTimeout()` schedules the callback for later.

JavaScript must finish synchronous code first.

Simple meaning:

> `setTimeout(..., 0)` means “run later as soon as possible,” not “run immediately.”

---

53. Microtasks and Macrotasks

Async callbacks are not all handled the same way.

Two important categories:

Category	Examples	Priority
Microtask	Promise `.then()`, resolved Promise callbacks	Runs before timer callbacks after sync code
Task / Macrotask	`setTimeout()`, `setInterval()`, some browser events	Runs after microtasks

Simple beginner memory:

```
sync code first
Promise callbacks next
timer callbacks after that
```

---

54. Event Loop Example

Study this code:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Output:

```
A
D
C
B
```

Step-by-step:

Step	What Happens
1	`A` runs immediately

Step	What Happens
2	`setTimeout()` schedules `B` for later
3	`Promise.resolve().then(...)` schedules `C` as a microtask
4	`D` runs immediately
5	Synchronous code is finished
6	Microtasks run next, so `C` runs
7	Timer tasks run after that, so `B` runs

Beginner rule:

> Normal code runs first. Promise callbacks usually run before timer callbacks.

Part N: Promise Tools

55. Why Promise Tools Matter

Sometimes you have more than one async task.

Examples:

- load user data and posts
- load multiple files
- call two APIs
- try multiple servers
- wait for several independent requests

Promise tools help manage multiple async operations.

---

56. `Promise.all()`

`Promise.all()` waits for all Promises to succeed.

Example:

```
const userPromise = fetch("/user.json");
const postsPromise = fetch("/posts.json");
```

```
const responses = await Promise.all([userPromise, postsPromise]);
```

Simple meaning:

> Wait until every Promise succeeds.

If one Promise fails, the whole `Promise.all()` fails.

Use when:

```
I need everything to succeed.
```

Example with JSON parsing:

```
async function loadDashboard() {
  try {
    const [userResponse, postsResponse] = await Promise.all([
      fetch("/user.json"),
      fetch("/posts.json")
    ]);

    if (!userResponse.ok || !postsResponse.ok) {
      throw new Error("One request failed");
    }

    const [user, posts] = await Promise.all([
      userResponse.json(),
      postsResponse.json()
    ]);

    console.log(user, posts);
  } catch (error) {
    console.error(error.message);
  }
}
```

---

57. `Promise.allSettled()`

`Promise.allSettled()` waits for all Promises to finish, even if some fail.

Example:

```
const results = await Promise.allSettled([
  fetch("/user.json"),
  fetch("/posts.json")
]);

console.log(results);
```

Each result tells you whether that Promise was fulfilled or rejected.

Simple meaning:

> Wait for everything to finish, then inspect what worked and what failed.

Use when:

```
I want to know what succeeded and what failed.
```

Example result shape:

```
[
  { status: "fulfilled", value: responseObject },
  { status: "rejected", reason: errorObject }
]
```

---

58. `Promise.race()`

`Promise.race()` finishes when the first Promise settles.

Example:

```
const fastest = await Promise.race([
  fetch("/server-a.json"),
  fetch("/server-b.json")
]);

console.log(fastest);
```

Simple meaning:

> Whichever Promise finishes first wins.

Use when:

```
I only care which one finishes first.
```

Important:

The first Promise can succeed or fail.

So you still need error handling.

---

59. Promise Tool Comparison

Tool	Waits For	Fails When	Good For
`Promise.all()`	All Promises succeed	Any Promise rejects	Required group of requests
`Promise.allSettled()`	All Promises finish	Does not fail as a group	Mixed success/failure results
`Promise.race()`	First Promise settles	First settled Promise rejects	Fastest result or timeout-style logic

Beginner rule:

> Use `Promise.all()` when all results are required. Use `Promise.allSettled()` when partial results are acceptable. Use `Promise.race()` when only the first result matters.

Part O: Geolocation API

60. What Is the Geolocation API?

The Geolocation API lets the browser ask for the user's location.

Simple meaning:

> Geolocation asks the browser for latitude and longitude, but the user must usually give permission.

Basic example:

```
navigator.geolocation.getCurrentPosition(position => {
  console.log(position.coords.latitude);
  console.log(position.coords.longitude);
});
```

---

61. `navigator.geolocation.getCurrentPosition()`

Basic syntax:

```
navigator.geolocation.getCurrentPosition(successCallback, errorCallback);
```

Part	Meaning
`navigator`	Browser/device information object
`geolocation`	Location-related API
`getCurrentPosition()`	Ask for current location
success callback	Runs if location is allowed and found
error callback	Runs if location fails or permission is denied

---

62. Geolocation Success Example

```
navigator.geolocation.getCurrentPosition(position => {
  console.log("Latitude:", position.coords.latitude);
  console.log("Longitude:", position.coords.longitude);
});
```

The `position` object contains coordinate data.

Common values:

```
position.coords.latitude
position.coords.longitude
position.coords.accuracy
```

Simple meaning:

If the user allows location access, the browser gives your callback a position object.

---

63. Geolocation with Error Handling

Better version:

```
navigator.geolocation.getCurrentPosition(
  position => {
    console.log("Latitude:", position.coords.latitude);
    console.log("Longitude:", position.coords.longitude);
  },
  error => {
    console.error("Could not get location:", error.message);
  }
);
```

Why this is better:

- the user may deny permission
- the device may not support location
- the browser may fail to get the location
- the location request may time out

Beginner rule:

Always plan for permission denial.

---

64. Geolocation DOM Example

HTML:

```
<button id="locationBtn">Get location</button>
<p id="locationStatus" aria-live="polite">Location not requested.</p>
```

JavaScript:

```
const locationBtn = document.getElementById("locationBtn");
const locationStatus = document.getElementById("locationStatus");

locationBtn.addEventListener("click", () => {
  if (!navigator.geolocation) {
```

```
    locationStatus.textContent = "Geolocation is not supported.";
    return;
  }

  locationStatus.textContent = "Requesting location...";

  navigator.geolocation.getCurrentPosition(
    position => {
      const latitude = position.coords.latitude;
      const longitude = position.coords.longitude;

      locationStatus.textContent = `Latitude: ${latitude}, Longitude: $
{longitude}`;
    },
    error => {
      locationStatus.textContent = "Could not get location.";
      console.error(error.message);
    }
  );
});
```

This combines:

- DOM selection
- click events
- feature detection
- async callback behavior
- permission error handling
- accessible status updates

Part P: Minimum Sprint 26 Code to Understand

65. Core Async Fetch Example

You should be able to explain this line by line:

```
async function loadUsers() {
  try {
    const response = await fetch("/users.json");

    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }
```

```
    const users = await response.json();
    console.log(users);
  } catch (error) {
    console.error(error.message);
  }
}

loadUsers();
```

Line-by-line explanation:

Code	Simple Meaning
`async function loadUsers()`	Creates a function that can use `await`
`try`	Starts a block where something might fail
`await fetch("/users.json")`	Starts a request and waits for the response inside this function
`if (!response.ok)`	Checks whether the server response was successful
`throw new Error(...)`	Creates an error if the request failed
`await response.json()`	Parses JSON into JavaScript data
`console.log(users)`	Prints the loaded data
`catch (error)`	Handles network errors, bad responses, or parsing problems
`loadUsers()`	Runs the async function

Simple meaning:

> This function loads users safely and handles failure instead of assuming everything works.

Part Q: Common Beginner Mistakes

66. Mistake: Thinking Async Code Runs Immediately

Problem:

```
let users;

fetch("/users.json")
  .then(response => response.json())
  .then(data => {
```

```
    users = data;
  });

console.log(users);
```

The output may be:

```
undefined
```

Why?

Because `console.log(users)` runs before the fetch finishes.

Better:

```
fetch("/users.json")
  .then(response => response.json())
  .then(users => {
    console.log(users);
  });
```

Beginner rule:

> Use async results inside the async handler or after `await`.

---

67. Mistake: Forgetting `await`

Problem:

```
async function loadUsers() {
  const response = fetch("/users.json");
  const users = response.json();

  console.log(users);
}
```

This is wrong because `fetch()` returns a Promise.

Better:

```
async function loadUsers() {
  const response = await fetch("/users.json");
  const users = await response.json();

  console.log(users);
}
```

Beginner rule:

> If you need the resolved result of a Promise inside an async function, use `await`.

---

68. Mistake: Using `await` Outside an Async Function

Problem:

```
const response = await fetch("/users.json");
```

In many beginner script contexts, this causes an error because `await` must be inside an `async` function.

Better:

```
async function loadUsers() {
  const response = await fetch("/users.json");
  const users = await response.json();
  console.log(users);
}

loadUsers();
```

Beginner rule:

> Put `await` inside an `async` function.

---

69. Mistake: Not Checking `response.ok`

Problem:

```
const response = await fetch("/wrong-url");
const data = await response.json();
```

This assumes the request worked.

Better:

```
const response = await fetch("/wrong-url");

if (!response.ok) {
  throw new Error(`Request failed: ${response.status}`);
}

const data = await response.json();
```

Beginner rule:

> Always check whether the response was successful before trusting the data.

---

70. Mistake: No Error Message for the User

Problem:

```
catch (error) {
  console.error(error.message);
}
```

This logs the error for the developer, but the user sees nothing.

Better:

```
catch (error) {
  status.textContent = "Could not load data.";
  console.error(error.message);
}
```

Beginner rule:

> Log useful details for developers, but also show a simple message to the user.

---

71. Mistake: Using `innerHTML` for User or Server Data

Risky:

```
li.innerHTML = user.name;
```

Safer:

```
li.textContent = user.name;
```

Why?

`textContent` treats the value as text.

`innerHTML` treats the value as HTML.

Beginner rule:

> Use `textContent` for user-controlled or server-controlled text unless you have a strong reason not to.

---

72. Mistake: Not Handling Geolocation Denial

Weak:

```
navigator.geolocation.getCurrentPosition(position => {
  console.log(position.coords.latitude);
});
```

Better:

```
navigator.geolocation.getCurrentPosition(
  position => {
    console.log(position.coords.latitude);
  },
  error => {
    console.error(error.message);
  }
);
```

Beginner rule:

> Any permission-based API needs a failure plan.

Part R: Dry Runs

73. Dry Run 1: Timer Order

Code:

```
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 1000);

console.log("End");
```

Expected output:

```
Start
End
Timer
```

Reason:

- `Start` runs immediately.
- Timer is scheduled.
- `End` runs immediately.
- Timer callback runs later.

---

74. Dry Run 2: Promise vs Timer

Code:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);
```

```
Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Expected output:

```
A
D
C
B
```

Reason:

- Synchronous code: `A`, `D`
- Microtask: `C`
- Timer task: `B`

---

75. Dry Run 3: `async/await` Order

Code:

```
async function test() {
  console.log("Inside 1");
  await Promise.resolve();
  console.log("Inside 2");
}

console.log("Outside 1");
test();
console.log("Outside 2");
```

Expected output:

```
Outside 1
Inside 1
Outside 2
Inside 2
```

Reason:

- `Outside 1` runs.
- `test()` starts.
- `Inside 1` runs.
- `await` pauses the rest of `test()`.
- `Outside 2` runs.
- Promise continuation runs later.
- `Inside 2` runs.

Part S: Sprint 26 Practice Plan

76. Practice Tasks

Do these in order.

Task 1: Predict Output

Predict the output before running:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Expected output:

```
A
C
B
```

Task 2: Create a Promise

Write a Promise that resolves after 1 second:

```
const wait = new Promise(resolve => {
  setTimeout(() => {
```

```
    resolve("Finished waiting");
  }, 1000);
});

wait.then(message => {
  console.log(message);
});
```

Task 3: Rewrite with `async/await`

```
function waitOneSecond() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("Finished waiting");
    }, 1000);
  });
}

async function run() {
  const message = await waitOneSecond();
  console.log(message);
}

run();
```

Task 4: Fetch JSON and Print It

```
async function loadData() {
  const response = await fetch("/data.json");
  const data = await response.json();
  console.log(data);
}

loadData();
```

Task 5: Add `try/catch`

```
async function loadData() {
  try {
```

```
    const response = await fetch("/data.json");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Could not load data:", error.message);
  }
}

loadData();
```

Task 6: Add `response.ok`

```
async function loadData() {
  try {
    const response = await fetch("/data.json");

    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }

    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("Could not load data:", error.message);
  }
}

loadData();
```

Task 7: Show Loading, Success, and Error in the DOM

```
<button id="loadBtn">Load data</button>
<p id="status" aria-live="polite">Idle</p>
```

```
const loadBtn = document.getElementById("loadBtn");
const status = document.getElementById("status");

async function loadData() {
  try {
    status.textContent = "Loading...";
```

```
    const response = await fetch("/data.json");

    if (!response.ok) {
      throw new Error(`Request failed: ${response.status}`);
    }

    const data = await response.json();
    console.log(data);

    status.textContent = "Data loaded.";
  } catch (error) {
    status.textContent = "Could not load data.";
    console.error(error.message);
  }
}

loadBtn.addEventListener("click", loadData);
```

Task 8: Try `Promise.all()`

```
async function loadBoth() {
  const [userResponse, postsResponse] = await Promise.all([
    fetch("/user.json"),
    fetch("/posts.json")
  ]);

  const [user, posts] = await Promise.all([
    userResponse.json(),
    postsResponse.json()
  ]);

  console.log(user, posts);
}

loadBoth();
```

Task 9: Add Geolocation

```
navigator.geolocation.getCurrentPosition(
  position => {
    console.log(position.coords.latitude);
    console.log(position.coords.longitude);
```

```
  },
  error => {
    console.error(error.message);
  }
);
```

Task 10: Explain `async` vs `defer`

Answer in your own words:

```
defer waits until HTML parsing is finished and keeps script order.
async runs as soon as the script is downloaded and does not guarantee order.
```

Part T: Sprint 26 Checkpoint Questions

77. Questions to Answer Without Notes

You understand Sprint 26 when you can answer these:

1. What is synchronous code?
2. What is asynchronous code?
3. Why does asynchronous JavaScript exist?
4. What is a callback?
5. Why does `setTimeout(..., 0)` not run before normal synchronous code?
6. What is a Promise?
7. What are the three Promise states?
8. What does `.then()` do?
9. What does `.catch()` do?
10. What does `.finally()` do?
11. What does `async` do?
12. What does `await` do?
13. Why must `await` usually be inside an `async` function?
14. What does `fetch()` return?
15. What does `response.json()` do?
16. Why should you check `response.ok`?
17. What is the difference between `.then()` and `async/await`?
18. What HTTP method reads data?
19. What HTTP method creates data?
20. Why do you use `JSON.stringify()` in a POST request?
21. What is the difference between `async` and `defer` script loading?
22. What is the difference between the JavaScript engine and runtime?

23. What is the event loop?
24. What usually runs first: Promise callbacks or timer callbacks?
25. What does `Promise.all()` do?
26. What does `Promise.allSettled()` do?
27. What does `Promise.race()` do?
28. What does `navigator.geolocation.getCurrentPosition()` do?
29. Why should geolocation code handle permission denial?
30. Why should async UI code show loading and error states?

Part U: Glossary

78. Key Terms

Term	Simple Meaning
synchronous	Runs line by line, one thing at a time
asynchronous	Starts work now and handles the result later
callback	A function passed into another function to run later
Promise	Object representing a future success or failure
pending	Promise is still waiting
fulfilled	Promise succeeded
rejected	Promise failed
`.then()`	Handles Promise success
`.catch()`	Handles Promise failure
`.finally()`	Runs after success or failure
`async`	Creates a function that returns a Promise and can use `await`
`await`	Waits for a Promise inside an async function
`fetch()`	Makes a network request
response object	Object returned by `fetch()` with status and body tools
`response.ok`	Boolean that says whether the response was successful
`response.status`	HTTP status code
`response.json()`	Converts JSON response body into JavaScript data
`try/catch`	Handles code that might fail

Term	Simple Meaning
HTTP method	Request action type, like GET, POST, PUT, PATCH, DELETE
GET	Read data
POST	Create data
PUT	Replace data
PATCH	Partially update data
DELETE	Remove data
`JSON.stringify()`	Convert JavaScript data into JSON text
`defer`	Load script while parsing HTML, run after HTML parsing, keep order
`async` script	Load script while parsing HTML, run as soon as downloaded, no order guarantee
JavaScript engine	Reads and executes JavaScript
runtime	Environment that provides extra APIs
event loop	Coordinates sync code and async callbacks
microtask	High-priority async callback category, such as Promise callbacks
macrotask/task	Async callback category, such as timer callbacks
`Promise.all()`	Wait for all Promises to succeed
`Promise.allSettled()`	Wait for all Promises to finish, success or failure
`Promise.race()`	Finish when the first Promise settles
Geolocation API	Browser API for requesting user location
permission denial	When the user refuses browser permission

Final Sprint 26 Summary

Sprint 26 teaches one major idea:

> JavaScript can start slow work, continue running other code, and come back when the slow work finishes.

You use:

- callbacks for functions that run later
- Promises to represent future results

- `.then()`, `.catch()`, and `.finally()` to handle Promise chains
- `async/await` to write cleaner async code
- `fetch()` to make network requests
- `response.ok` and `try/catch` to handle failures
- HTTP methods to connect CRUD to servers
- `JSON.stringify()` to send JavaScript data as JSON
- `defer` and `async` to control script loading
- event loop knowledge to understand execution order
- Promise tools to manage multiple async operations
- Geolocation API to request user location safely

Core beginner sentence:

> Sprint 26 is about writing JavaScript that can wait without freezing, fail without crashing, and update the page clearly when async work finishes.